National University of Computer and Emerging Sciences



Lab Manual 06
Computer Organization
and Assembly Language

Course Instructor	Mr. Salman Mubarik
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Fall 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Objectives: 3

Activities: 3

Activity 1 (Assembly Language Tasks): 3

1) Factorial Using a Subroutine 3
2) Calculate a Power Series 3
3) Analyze a Stack Frame 4
4) Subtract Using Stack Parameters 4
5) Nested Subroutine Calls 5
6) Merge Two Sorted Arrays 5
7) Sum of Two Numbers Using a Stack Function 5

Objectives:

- Understand the concept, purpose and use of subroutines/functions
- Understand and use the stack for temporary storage and parameter passing.

Activities:

Activity 1 (Assembly Language Tasks):

1) *Factorial Using a Subroutine*

Write an Assembly Language program containing a subroutine to calculate the factorial of a number. The subroutine should:

- Receive the number as a parameter through the stack.
- Calculate the factorial.
- Use a separate multiplication subroutine.
- Do not use the MUL instruction.
- Return the factorial result through the stack.
- The caller should retrieve the returned value from the stack.

Example

For:

n = 5

the result should be:

5! = 5 × 4 × 3 × 2 × 1
= 120
= 0078h

Requirements

- Use PUSH and POP.
- Use CALL and RET.
- Pass the parameter through the stack.
- Use BP to access the parameter.
- Implement multiplication using repeated addition.
- Properly preserve registers used by the caller.

2) *Calculate a Power Series*

Write a program to calculate:

$$ans = \sum_{n=1}^{l} r^n$$

where r and l are passed to the main subroutine through the stack.

Example

If:
r = 2
l = 8
then:

ans = $2^1 + 2^2 + 2^3 + 2^4 + 2^5 + 2^6 + 2^7 + 2^8$
= 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256
= 510
= 01FEh

Requirements

- Pass r and l through the stack.
- Create multiple subroutines.
- Parameters passed between subroutines must also be passed through the stack.
- Use MUL for multiplication.
- Return the final answer through the stack.
- Do not use global variables.
- Save and restore registers where necessary.

3) Analyze a Stack Frame

Consider the following stack arrangement for a subroutine:

Return Value
4 Parameters
2 Local Variables
4 Saved Register Copies

Answer the following questions.

(i) How can the subroutine access all four parameters?
(ii) How can the subroutine place its return value on the stack?
(iii) How can the subroutine access its local variables?
(iv) How can the stack be cleaned before the subroutine returns?
(v) How can the caller pass parameters and retrieve the return value?

Requirement

For each answer, draw the corresponding stack diagram and identify the appropriate BP offsets.

4) Subtract Using Stack Parameters

Implement a subroutine:

Subtract(int a, int b, int c, int d)

that performs:

Result = a – b – c – d

The parameters must be passed through the stack. The subroutine must:

- Create a stack frame using BP.
- Access parameters using [BP+offset].
- Calculate the result.
- Return the result.
- Properly clean the stack before returning.
- Preserve and restore registers used by the caller.
- Do not hard-code separate programs for the three test cases.

5) *Nested Subroutine Calls*

Implement:

AnotherFunction(int a, int b, int c, int d)

with the following logic:

```
localVar1 = Subtract(a,b,c,d)
localVar2 = Subtract(c,d,0,0)
return localVar1 + localVar2
```

Requirements

This task must demonstrate:

- Stack-based parameter passing
- Return values
- Local variables
- Nested function calls
- Stack frames
- BP-based parameter access
- Register preservation
- Proper stack cleanup
- No global variables

Students should inspect the stack using AFD and identify the nested activation records.

6) *Merge Two Sorted Arrays*

Write a subroutine:

MergeArrays

that merges two already sorted arrays of words into one sorted array.

Input

A1: [2, 5, 9]
A2: [1, 4, 7, 8]

Output

Merged: [1, 2, 4, 5, 7, 8, 9]

Requirements

- Both input arrays are already sorted in ascending order.
- Do not sort the input arrays first.
- Compare elements from both arrays.
- Copy the smaller element to Merged.
- Continue until one array is exhausted.
- Copy the remaining elements of the other array.
- Implement the merging logic inside a subroutine.
- Pass required information to the subroutine through the stack.

7) *Sum of Two Numbers Using a Stack Function*

Write a subroutine that calculates the sum of two integers. The function should:

- Accept two parameters.
- Parameters may be passed through the stack.

- Use PUSH and POP.
- Use CALL and RET.
- Use BP to access parameters.
- Return the result in AX.
- Properly clean the stack.
- Preserve registers where required.

Example

a = 25

b = 17

Expected:

25 + 17 = 42

In hexadecimal:

19h + 11h = 2Ah